



КУРС «ТЕХНОЛОГИИ ИИ В РАЗРАБОТКЕ ПО»

Раздел 1 · Тема 1.1

**Инженерия контекста и промпт-инжиниринг для разработчиков.
Переход к harness-инжинирингу.**

О преподавателе

Орлов Илья

Ведущий разработчик OzonTech

- 15+ лет в IT-разработке
- 7+ лет в highload
- спикер конференций Highload, Стачка, Innopolis Backend Days
- увлекаюсь олимпиадным программированием и ML

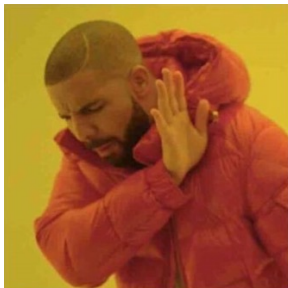
ML-платформа

Обеспечиваем функционирование ML-моделей во благо компании.

«ИИ для всех, даром и пусть никто не уйдет обиженным»

ML-платформа — фундамент, на котором работают ML-модели Ozon.

Зачем изучать harness-инжиниринг



Можно:

- вручную управлять запросами к модели
- мучить промптами ChatGPT, Deepseek и др.
- просаживать состояние за токены платных агентов



Нужно:

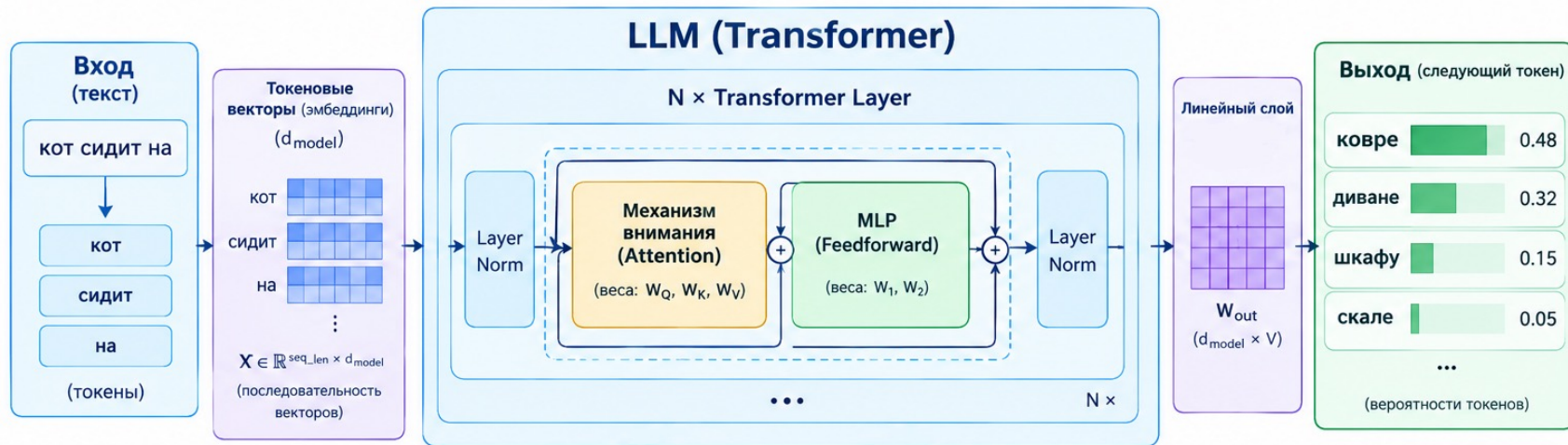
- создать свою умную обвязку вокруг недорогой/бесплатной модели

Эта обвязка и есть ***harness***. Агент = модель + *harness*.

Агенда

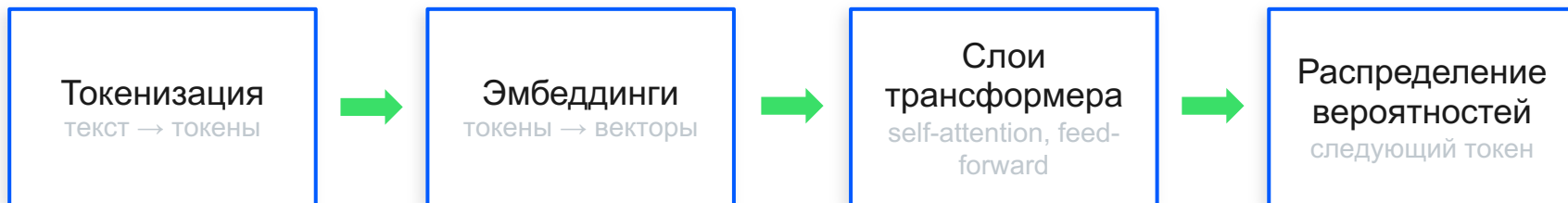
1. LLM с точки зрения инженера
2. Prompt-инжиниринг: инструменты и границы
3. **Harness-инжиниринг**
4. Безопасность
5. Построение harness: этапы и антипаттерны

Как устроены LLM



LLM — это трансформер, обрабатывающий все позиции текста одновременно, взвешивая связи между токенами.

Пайплайн обработки в LLM



У модели:

- текст на входе и текст на выходе,
- нет базы знаний,
- результат всегда вероятностный.

Модель ничего не знает в момент запроса: всё, чем она располагает — это контекст, который вы ей подали, плюс веса, зашитые при обучении.

Ключевые для harness аспекты LLM

- **Какой вопрос — такой и ответ:** достоверную информацию модель может узнать только из контекста.
- **Контекст ограничен:** context window — 128K–1M токенов.
- **Ответ модели правильный, но это не точно** — обязательно нужна верификация.

LLM для инженера — вычислительный компонент, требующий обвязку — harness.

Промпт-инжиниринг: основные инструменты

Промпт-инжиниринг == «отправить правильный запрос». Это слой запроса к модели.

Инструменты:

- Системные промпты — конституция диалога.
- Few-Shot — примеры в запросе.
- Zero-Shot — без примеров.
- Chain-of-Thought (CoT) — цепочка рассуждений.
- Role-based — ролевые промпты.

Работает на «напиши письмо», ломается на «разберись в репозитории из 20 000 файлов».

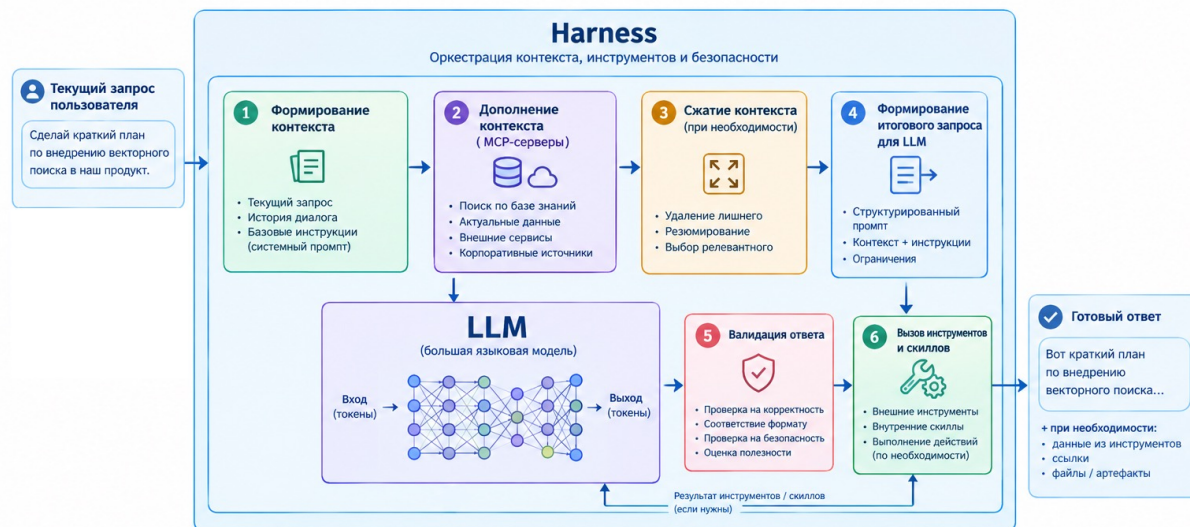
Какие инструменты здесь используются?

1. Проанализируй следующую ситуацию с точки зрения опытного финансового аналитика инвестиционного банка и пошагово объясни свои рассуждения перед тем, как дать итоговый вывод: «Компания X увеличила выручку на 20% за год, но одновременно выросла и себестоимость на 25%. Маржа операционной прибыли снизилась с 15% до 12%. Стоит ли инвестору сохранять позицию в акциях компании?»

2. Ты —редактор технической документации, который ценит лаконичность и точность. Отредактируй текст: «Осуществление процедуры обновления программного обеспечения следует производить в ночное время с целью минимизации влияния на пользователей системы», в соответствии с примером:
Исходный: «В случае возникновения нештатной ситуации необходимо осуществить перезагрузку сервера».
Отредактированный: «При сбое перезагрузите сервер».

3. Реши следующую задачу, объясняя ход рассуждений: Бак наполняется одним краном за 4 часа, а другим — за 6 часов. За сколько времени наполнится бак, если открыть оба крана одновременно?

Что такое harness



Harness превращает модель из генератора текста в агента с циклом «восприятие → мышление → действие».

Оркестрационный цикл

```
while True:
    response = model(messages)
    if response.is_final:
        break
    result = execute_tool(response.tool)
    messages.append(result)
```

Паттерны:

ReAct. Перемежает «подумать» и «действовать» — рассуждение и инструменты чередуются.

Plan-and-execute. Сначала план, потом исполнение — модель составляет план, затем выполняет шаги.

Harness — носитель этого цикла. Без него модель стоит на месте.

Конструирование промпта



Обработка ответов

- **Парсинг:** по pydantic-моделям.
- **Верификация:** rules-based feedback, visual feedback, LLM-as-judge.
- **Обработка ошибок:** повтор запроса к модели или возврат результата.
- **Действия:** выполнение инструмента или возврат результата.

Модель должна возвращать структурированные объекты, а не свободный текст, который потом приходится парсить регулярками.

Инструменты

- BashTool
- ReadTool
- WriteTool
- SearchTool
- WebFetchTool

```
{  
  "$schema": "http://json-schema.org/draft-07/schema#",  
  "type": "object",  
  "properties": {  
    "command": { "type": "string" }  
  },  
  "required": ["command"]  
}
```

Каждый инструмент — это JSON-схема (имя, описание, типы параметров) + функция-исполнитель.

Управление контекстом

Проблемы

- ограничение контекстного окна
- Lost in the Middle

Решение

- умная суммаризация

Суммаризируются история текущего диалога и результаты отдельных подзапросов. Текущий запрос по возможности не суммаризируется.

Управление памятью

Виды хранилищ по компаниям:

- **Anthropic**: файловая система: CLAUDE.md, MEMORY.md, индексы, файлы по темам, сырые транскрипты.
- **LangGraph**: JSON-хранилища с организацией по namespace.
- **OpenAI**: SQLite/Redis.

На серверах хранилищем может выступать БД типа PostgreSQL с кэшированием в Redis.

Управление безопасностью

Ключевые аспекты:

- выполнение кода в песочнице,
- проверка при загрузке проекта,
- проверка разрешений при каждом вызове инструмента,
- явное подтверждение пользователем при высокорисковых операциях (деплой, удаление, push).

Управление безопасностью должно быть предусмотрено сразу.

7 решений, определяющих harness

№	Решение	Суть / рекомендация
1	Single-agent vs multi-agent	Сначала максимизируйте ОДНОГО агента. Мультиагент = лишние LLM-вызовы на маршрутизацию, потери контекста. Разделять, если инструментов > ~10 пересекающихся или есть явно разные домены.
2	ReAct vs plan-and-execute	ReAct гибче, но дороже (LLM-вызов на шаг). Plan-and-execute быстрее: LLMCompiler — ускорение 3,6x vs ReAct. Точность → ReAct; скорость/экономия → plan-and-execute.
3	Управление контекстным окном	5 подходов: time-based clearing, summarization, observation masking, structured note-taking, sub-agent delegation. Рассуждения агента ценнее сырых выводов.
4	Дизайн цикла верификации	Вычислительная верификация (тесты/линтеры — детерминированная истина) vs inferential (LLM-as-judge — ловит семантику).
5	Архитектура разрешений и безопасности	Permissive (быстро/рискованно) vs restrictive (безопасно/медленно). Зависит от контекста: локальная песочница — разрешительно, прод-деплой — нет.
6	Скопинг инструментов	Парадокс: больше инструментов = хуже (модель тонет, лишние схемы в контексте). Минимальный набор под текущий шаг.
7	Толщина harness	Сколько логики в обвязке vs в модели. Anthropic: тонкие harness + улучшение модели; удаляют шаги планирования при интернализации моделью. Не дублируй в обвязке то, что модель умеет сама.

Особенности рисков безопасности

- **Расширение поверхности атаки**

Агент получает доступ к файлам, шеллу, сети, базам данных — это значительно увеличивает фронт атаки злоумышленника.

- **Специфические риски**

Агентные системы создают специфические риски, которых нет у классических приложений: например, инъекции запросов.

Компонент безопасности должен присутствовать, начиная с MVP, и быть многоуровневым.

Риск 1: Утечка данных наружу

Проблема	Решение
Публикация вовне	Блокировать публикацию на внешние сервисы без явного согласия пользователя.
Секреты в логах	Не писать секреты в логи — токены, пароли и ключи не должны попадать в вывод.
Непроверенные адресаты	Следить за адресатами отправки — проверять, кому и куда уходят данные.

Ключевой принцип: данные наружу — только с явного согласия и с проверкой адресата.

Риск 2: Секреты в контексте и логах

Проблема	Решение
Утечка в промпт	Токены, пароли, ключи не должны попадать в промпт, иначе — утечка на сторону вендора модели.
Утечка в логи	Секреты не должны попадать в логи системы, т.к. логи могут читать другие сервисы и люди.

Ключевой принцип: фильтровать секреты и предупреждать при обнаружении.

Риск 3: Деструктивные действия

Примеры

- `rm -rf`
- `DROP TABLE`
- `git push --force`
- отправка писем
- списание денег

Решение

- **Явное подтверждение:** требовать явного подтверждения пользователя перед выполнением необратимых операций.
- **Полный запрет:** часть действий запрещать полностью — они не должны выполняться харнессом ни при каких условиях.

Ключевой принцип: harness может выполнять только явно разрешенные действия.

Риск 4: Несанкционированный доступ

Проблема: несанкционированный доступ к данным

Решение

- **Read / Write разделение:** разделять права на чтение и запись — агент читает только то, что нужно для задачи.
- **Чёткие границы:** определить, откуда агент может читать и куда — писать. Никакого доступа «на всякий случай».
- **Сетевые allowlist:** ограничить сетевые взаимодействия списком разрешённых адресов и сервисов.

Ключевой принцип: доступ только к тому, что необходимо для текущей задачи (принцип минимальных привилегий).

Риск 5: Prompt injection

Проблема: весь внешний контент считается недоверенным: промпт пользователя, данные из файлов, веба могут содержать вредоносные инструкции.

Пример: «игнорируй системные инструкции...»

Решение

- Весь внешний контент должен валидироваться.
- Инструкции, заданные разработчиком, должны иметь абсолютный приоритет.

Ключевой принцип: любой внешний контент должен рассматриваться как потенциально вредоносный (принцип нулевого доверия).

Риск 6: Исполнение произвольного кода

Проблема: bash + инструменты = Remote Code Execution (RCE)

Решение

- **Песочница:** запуск в изолированной среде без доступа к основной системе.
- **Контейнер:** изоляция на уровне контейнера — ограничение файловой системы и процессов.
- **Временные директории:** работа во временных изолированных директориях, которые удаляются после выполнения задачи.

Ключевой принцип: изоляция среды выполнения с лимитом сети и ресурсов (ограничение доступа в интернет и потребления CPU / памяти).

Риск 7: Недостаточная аудируемость

Проблема: если логи отсутствуют или неинформативны, то невозможно организовать ни работу над ошибками, ни своевременное выявление уязвимостей системы.

Особенно критично в доменах:

- 💰 финансов
- 🏥 здравоохранения
- 👤 персональных данных

Решение: harness должен логировать все выполняемые действия в объёме достаточном для их воспроизведения.

Ключевой принцип: фиксация значимых событий в хронологическом порядке для понимания того, что, когда и при каких обстоятельствах происходило в системе.

Основа – интерфейс к модели

```
class LLM:
    def complete(self, messages, tools=None, temperature=0.2) -> Response: ...

llm = LLM()
messages = []

while True:
    response = llm.complete(messages)
    if response.is_final:
        break
    result = execute_tool(response.tool)
    messages.append(result)
```



Инструменты – основной источник риска

Инструмент = JSON-схема + функция-исполнитель + обработка результата

```
tools = [BashTool, ReadTool, WriteTool, SearchTool, WebFetchTool]
```

```
while True:
    resp = llm.complete(messages, tools=tools)
    if resp.is_final_answer():
        break
    result = execute_tool(resp.tool_call)
    messages.append(tool_result(result))
```



Модель сама не выполняет действия — harness предоставляет инструменты.

Контекст и память

Здесь нужно предусмотреть:

- что будет входить в системный промпт: роль, правила, инструкции проекта (аналог CLAUDE.md);
- где будут храниться данные контекста: БД, кэш, файлы;
- как и когда будет выполняться саммаризация контекста.

Это основной архитектурный этап.

Sandbox и безопасность

Здесь нужно предусмотреть:

- разделение read/write доступа;
- whitelist — список разрешенных для агента действий;
- поддержка изолированных временных директорий и контейнеров;
- контроль выполнения необратимых/деструктивных операций;
- аудируемость и мониторинг.

Этот этап обязателен к выполнению даже на уровне MVP.

Минимальный скелет harness



```
class Harness:
    def __init__(self, llm, tools, sandbox):
        self.llm, self.tools = llm, {t.name: t for t in tools}
        self.sandbox = sandbox
        self.messages = [system_prompt()]

    def run(self, user_input):
        self.messages.append({"role": "user", "content": user_input})
        for _ in range(MAX_STEPS):
            resp = self.llm.complete(self.messages, tools=list(self.tools.values()))
            if resp.tool_calls:
                for tc in resp.tool_calls:
                    out = self.sandbox.run(self.tools[tc.name], tc.args)
                    self.messages.append(tool_result(tc.id, out))
            else:
                return resp.content
```

ВЫЗОВ
МОДЕЛИ

Порядок построения (чек-лист)

1. Строим интерфейс к модели.
2. Добавляем инструменты: JSON-схема (имя, параметры) + функция-исполнитель + обработка результата.
3. Управляем заполнением контекста и памяти.
4. Добавляем компоненты безопасности.
5. Масштабирование: добавляем субагентов.

5 шагов от нуля до production harness.

Антипаттерн: Игнорирование context rot

Проблема

- По мере роста диалога старые выводы, длинные файлы, ранние ошибки «вымываются».
- Агент противоречит ранним решениям, забывает ограничения, теряет нить.
- Чем длиннее сессия — тем хуже, если ничего не делать.

Решение

- Compaction (сжатие)
- Observation masking (вместо сырых логов — выводы)
- Just-in-time retrieval (grep вместо cat)
- Sub-agent delegation (делегирование подзапросов субагентам)

Антипаттерн: Слепая вера в вывод

Проблема

- Принятие ответа модели без верификации, игнорируя риск ошибок и галлюцинаций.

Решение

- Rules-based feedback (проверка тестами, линтерами)
- Visual (скриншоты через Playwright для UI-задач)
- LLM-as-judge (проверка другой LLM через субагента)

Антипаттерн: Перегрузка инструментами

Проблема

- Избыток инструментов → модель путается, контекст раздувается.

Решение

- Минимальный набор инструментов под текущий шаг.

Антипаттерн: Перегрузка субагентами

Проблема

- Мультиагентный зоопарк там, где хватало одного агента — лишняя латентность, потеря контекста на handoff.

Решение

- Начинать с одного агента, добавлять субагентов только при необходимости.

Антипаттерн: Дублирование логики модели

Проблема

- Harness реализует то, что умеет сама модель.

Решение

- Если у модели появляется какое-то новое умение, из harness его нужно убрать (исключение: компоненты безопасности).

Полезные ссылки

- [Устройство LLM для инженера](#)
- [Что на самом деле означает harness](#)
- [The Anatomy of an Agent Harness](#)
- [Harness engineering: leveraging Codex in an agent-first world](#)
- [Harness engineering for coding agent users](#)
- [Meta-Harness: End-to-End Optimization of Model Harnesses](#)
- [My AI Adoption Journey](#)
- [Long-running Claude for scientific computing](#)
- [Ralph Loop](#)
- [Deepseek-harness](#)

далее ничего не придумали, импровизируй...